

Capítulo 4: Entradas y Salidas en C#

Consola estándar, captura de datos, parsing seguro, streams de entrada/salida y formateo profesional.

Curso: Desarrollo Profesional de Software en C#

Unidad: Capítulo 4

Nivel: Desde Principiante a Nivel Práctico

Enfoque: Aplicaciones de Consola y Sistemas de Control

1. CONECTANDO CONOCIMIENTOS

En el **Capítulo 2** aprendiste a declarar tipos de datos (como `int`, `double`, `string` y `bool`), y en el **Capítulo 3** dominaste las estructuras de variables y constantes en memoria. Sin embargo, tus programas hasta ahora han sido totalmente aislados: los datos estaban "hardcodeados" directamente en el código fuente. Para crear aplicaciones útiles, el programa debe comunicarse con el mundo exterior: recibir instrucciones del usuario, procesarlas y devolver un resultado claro.

2. OBJETIVO DEL CAPÍTULO

Este capítulo enseña la arquitectura de Entradas y Salidas (I/O) en el ecosistema .NET mediante la clase estática `System.Console`. Aprenderás a solicitar información por teclado, transformar texto en tipos numéricos con parsing seguro, formatear salidas complejas y construir interfaces interactivas para software industrial y de consola.

Al finalizar este capítulo serás capaz de:

- Comprender la arquitectura de los tres Streams estándar de .NET: `Console.In (StdIn)`, `Console.Out (StdOut)` y `Console.Error (StdErr)`.
- Capturar datos ingresados por teclado utilizando `Console.ReadLine()`, `Console.Read()` y `Console.ReadKey()`.
- Convertir cadenas de texto a tipos de datos primitivos utilizando `int.Parse()` y defensivamente mediante `int.TryParse()` para evitar interrupciones por excepción.
- Dominar el formateo de salida utilizando concatenación, formato compuesto (Composite Formatting) e interpolación de cadenas (`$""`).
- Diseñar sistemas interactivos profesionales con manejo de nulos, limpieza de pantalla (`Console.Clear()`) y códigos de colores (`ConsoleColor`).

3. INTRODUCCIÓN: ¿QUÉ SON LAS ENTRADAS Y SALIDAS (I/O)?

Imagina una calculadora física. Si la calculadora no tuviera botones (entradas) ni una pantalla (salida), por más potente que fuera su procesador interno, resultaría inútil. En C#, las **Entradas y Salidas** (Input/Output o **I/O**) representan los canales de comunicación entre la memoria de la aplicación y dispositivos externos como la pantalla, el teclado, un archivo o un puerto serie conectado a un sensor industrial.

4. EXPLICACIÓN DESDE CERO: TÉRMINOS FUNDAMENTALES

Para dominar el manejo de I/O en .NET sin vacíos conceptuales, es esencial comprender la terminología técnica del compilador y del sistema operativo:

1. System.Console

Clase estática del BCL (Base Class Library) de .NET en el espacio de nombres System que proporciona acceso directo a los flujos estándar de entrada, salida y error para aplicaciones basadas en terminal.

2. Stream (Flujo de Datos)

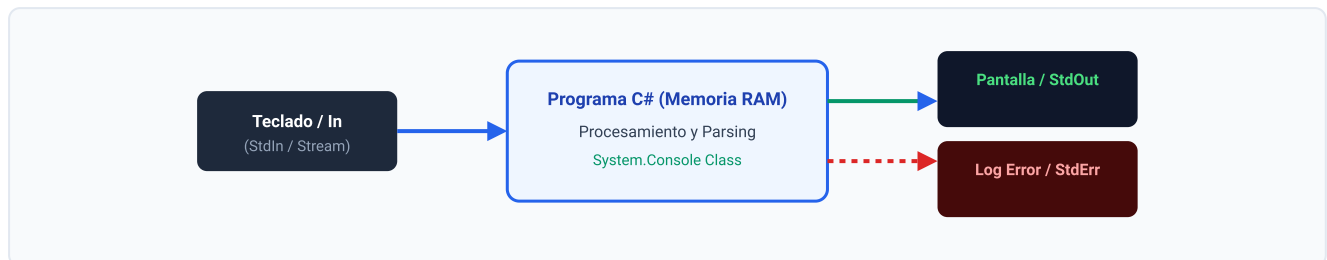
Secuencia continua de bytes enviada o recibida de forma secuencial entre el programa y un dispositivo de hardware o memoria. Abstrae la lectura/escritura física.

3. Standard Output (StdOut)

Flujo por defecto asignado a la consola (pantalla) para mostrar información normal del sistema. En C# se manipula vía `Console.Write()` y `Console.WriteLine()`.

4. Standard Input (StdIn)

Flujo por defecto que lee caracteres desde el teclado del sistema. Se manipula a través de métodos como `Console.ReadLine()` y `Console.ReadKey()`.



5. LA SALIDA ESTÁNDAR: MÉTODOS WRITE() Y WRITELINE()

Para proyectar información procesada hacia la pantalla se emplean dos métodos principales de la clase Console:

- `Console.Write(valor)`: Escribe la representación en cadena del valor en la posición actual del cursor sin agregar un salto de línea final.
- `Console.WriteLine(valor)`: Escribe el valor solicitado y añade automáticamente un carácter de retorno de carro y salto de línea (`\r\n` en Windows, `\n` en Linux/macOS) al final, desplazando el cursor al inicio de la siguiente fila.

6. LA ENTRADA ESTÁNDAR Y CAPTURA POR TECLADO

Capturar la entrada del usuario requiere detener temporalmente la ejecución del hilo principal de la aplicación hasta que el usuario envíe una señal física desde el teclado.

Método	Tipo de Retorno	Comportamiento Operativo	Uso Recomendado
<code>Console.ReadLine()</code>	<code>string?</code>	Lee una línea completa de caracteres ingresada por el usuario hasta que presiona la tecla Enter. Retorna un null en caso de EOF.	Captura de nombres, comandos, textos y lecturas generales de usuario.
<code>Console.ReadKey()</code>	<code>ConsoleKeyInfo</code>	Intercepta una sola tecla física presionada inmediatamente sin esperar que el usuario presione Enter.	Menús interactivos, pausar ejecución, confirmaciones (S/N) y videojuegos en consola.
<code>Console.Read()</code>	<code>int</code>	Lee únicamente el siguiente carácter del flujo de entrada y retorna su código entero Unicode (o -1 si finaliza).	Procesamiento avanzado de bajo nivel byte a byte o buffers de caracteres.

7. EL PROBLEMA DEL CONVERSOR DE TIPOS (PARSING)

Todo dato capturado por la consola mediante `Console.ReadLine()` ingresa a la memoria como una cadena de texto (`string`), **incluso si el usuario teclea números** como "45". Si intentas realizar operaciones aritméticas directas con el valor leído, el compilador C# arrojará un error de tipado estricto (CS0029).

Regla de Oro en C#:

El teclado SIEMPRE devuelve texto. Para realizar cálculos numéricos (sumas, porcentajes, comparaciones de umbrales) es estrictamente obligatorio realizar una conversión explícita o proceso de **Parsing**.

Mecanismos de Conversión: Parse vs. TryParse

Existen dos aproximaciones para transformar cadenas numéricas a tipos primitivos (`int`, `double`, `decimal`):

```
// APROXIMACIÓN FRÁGIL: int.Parse() // Si el usuario escribe "hola" en lugar de un número, la aplicación explota con FormatException
string entradaTexto = Console.ReadLine(); int edad = int.Parse(entradaTexto); // ¡Riesgo alto en producción!
// APROXIMACIÓN ROBUSTA Y DEFENSIVA: int.TryParse() // Retorna un 'bool' que indica si la conversión fue exitosa y asigna el valor vía parámetro 'out'
bool exito = int.TryParse(entradaTexto, out int edadSegura); if (exito) { Console.WriteLine($"Edad procesada correctamente: {edadSegura}"); } else { Console.WriteLine("Error: El texto ingresado no es un número entero válido."); }
```

8. FORMATEO PROFESIONAL DE SALIDA EN C#

Proyectar información no consiste solo en mostrar números en pantalla; implica presentarlos en formatos legibles según el contexto del usuario (monedas, decimales fijos, porcentajes o fechas).

Evolución de Técnicas de Formateo de Texto

1. Concatenación (+)

```
string m = "Temp: " + t + " °C";
```

Ineficiente en memoria por la inmutabilidad de los strings; código difícil de mantener.

2. Composite Formatting

```
Console.WriteLine("Temp: {0:F2} °C", t);
```

Usa marcadores posicionales indexados en cero con especificadores de formato opcionales.

3. Interpolación de Cadenas (\$) — Estándar Moderno C#

Permite incrustar expresiones C# directamente dentro de las llaves {} de una cadena, incluyendo especificadores de formato profesional precedidos por dos puntos .

```
double presionBar = 4.5892; decimal costoLote = 1250.75m; double porcentajeEficiencia = 0.942; Console.WriteLine($"Presión calibrada: {presionBar:F2} bar"); // Muestra: Presión calibrada: 4.59 bar Console.WriteLine($"Costo de producción: {costoLote:C2}"); // Muestra: Costo de producción: $1,250.75 Console.WriteLine($"Rendimiento del PLC: {porcentajeEficiencia:P1}"); // Muestra: Rendimiento del PLC: 94.2%
```

ESPECIFICADORES DE FORMATO FRECUENTES

Código	Nombre	Ejemplo de Entrada	Resultado Formateado
:C / :C2	Currency (Moneda)	1500.5m	\$1,500.50 (según Configuración Regional)
:F2 / :N2	Fixed-Point / Number	12.34567	12.35 (redondeo estándar)
:P1	Percentage (Porcentaje)	0.854	85.4%
:D5	Decimal Padded (Relleno Ceros)	42	00042 (ideal para números de lote/ID)

9. EJEMPLOS PROGRESIVOS DE CÓDIGO

Nivel 1: Ejemplo Básico (Consola e Interacción Básica)

```
// Captura básica de cadena y saludo personalizado Console.Write("Ingrese el nombre del operador del turno: "); string? nombreOperador = Console.ReadLine(); Console.WriteLine($" [SISTEMA] Operador registrado correctamente: {nombreOperador}");
```

Análisis Línea por Línea: Usamos `Console.Write()` para imprimir la solicitud sin salto de línea, permitiendo que el usuario escriba en la misma fila. Luego `ReadLine()` captura el texto tipeado y se emite el mensaje mediante interpolación.

Nivel 2: Ejemplo Intermedio (Parsing Seguro con Validación)

```
// Lectura segura de un parámetro numérico con bucle de reintento int cantidadPiezas; bool esValido = false; do { Console.Write("Ingrese la cantidad de piezas producidas (entero positivo): "); string? entrada = Console.ReadLine(); esValido = int.TryParse(entrada, out cantidadPiezas) && cantidadPiezas >= 0; if (!esValido) { Console.WriteLine(" [ERROR] Entrada no válida. Debe ingresar un número entero mayor o igual a 0."); } } while (!esValido); Console.WriteLine($" [ÉXITO] Lote de producción registrado: {cantidadPiezas} unidades.");
```

Nivel 3: Ejemplo Práctico (Interfaz de Telemetría para Sensor)

```
// Captura de múltiples parámetros de un sensor industrial de presión Console.WriteLine("=== MÓDULO DE CONFIGURACIÓN DE SENSOR DE PRESIÓN PLC ==="); Console.Write("Identificador de Sensor (ej. SENSOR-PLC-01): "); string idSensor = Console.ReadLine() ?? "DESCONOCIDO"; Console.Write("Lectura de Presión actual (Bar): "); double.TryParse(Console.ReadLine(), out double presionBar); Console.Write("Límite Crítico de Alarma (Bar): "); double.TryParse(Console.ReadLine(), out double limiteAlarmaBar); bool estadoAlerta = presionBar >= limiteAlarmaBar; Console.WriteLine("\n--- RESUMEN DE TELEMETRÍA ---"); Console.WriteLine($"ID Dispositivo: {idSensor.ToUpper()}"); Console.WriteLine($"Presión Medida: {presionBar:F2} Bar"); Console.WriteLine($"Estado Alarma : {(estadoAlerta ? "CRÍTICO (Supera límite)" : "NORMAL")}");
```

10. EJEMPLO PROFESIONAL (NIVEL 4: SISTEMA INTERACTIVO PLC)

El siguiente programa implementa una consola interactiva profesional con formateo visual, código de colores en pantalla (ConsoleColor) y procesamiento seguro de flujos I/O para supervisión de una planta de ensamblaje.

```
using System; namespace SistemaAutomatizacionIndustrial { class Program { static void Main(string[] args) { // Configuración del entorno gráfico de consola Console.Title = "Sistema HMI - Panel de Control de Tanque de Mezcla"; Console.Clear(); // Encabezado con colores de consola Console.ForegroundColor = ConsoleColor.Cyan; Console.WriteLine("====="); Console.WriteLine(" PANEL HMI - SISTEMA DE CONTROL DE FLUJO DE QUÍMICOS"); Console.WriteLine("====="); Console.ResetColor(); // Lectura e Interpolación de Parámetros Console.Write("\nIngrese el código del Lote de Producción: "); string codigoLote = Console.ReadLine() ?? "LOTE-0000"; Console.Write("Ingrese Volumen Objetivo de Mezcla (Litros): "); double.TryParse(Console.ReadLine(), out double volumenObjetivo); Console.Write("Ingrese Caudal de Inyección Actual (L/min): "); double.TryParse(Console.ReadLine(), out double caudalInyeccion); // Cálculo del tiempo estimado de llenado double tiempoMinutos = caudalInyeccion > 0 ? (volumenObjetivo / caudalInyeccion) : 0; // Salida Formateada de Resultados Console.WriteLine("\n-----"); Console.ForegroundColor = ConsoleColor.Yellow; Console.WriteLine($"REPORTE OPERATIVO - {codigoLote.ToUpper()}"); Console.ResetColor(); Console.WriteLine($"Volumen Solicitado : {volumenObjetivo:N2} L"); Console.WriteLine($"Flujo de Entrada : {caudalInyeccion:F1} L/min"); Console.WriteLine($"Tiempo Estimado : {tiempoMinutos:F2} Minutos"); if (tiempoMinutos > 60.0) { Console.ForegroundColor = ConsoleColor.Red; Console.WriteLine(" ALERTA: El tiempo de llenado excede el límite de turno (1 hora)."); } else { Console.ForegroundColor = ConsoleColor.Green; Console.WriteLine(" [OK] Parámetros operativos dentro del rango nominal."); } Console.ResetColor(); // Intercepción de tecla final para cerrar Console.WriteLine("\nPresione cualquier tecla para finalizar el monitoreo..."); Console.ReadKey(true); } } }
```

11. COMPARACIONES IMPORTANTES Y TABLAS DE DIFERENCIAS

Diferencias entre Métodos de Entrada por Teclado

Concepto	ReadLine()	ReadKey()	Read()
Retorno	Cadena de texto (string?)	Objeto estructura (ConsoleKeyInfo)	Entero numérico (int) Unicode
Espera tecla Enter	Sí, procesa al presionar Enter	No, intercepta inmediatamente al pulsar la tecla	Sí, espera al buffer de línea
Muestra en Pantalla	Sí, hace eco del texto tipeado	Configurable mediante parámetro intercept	Sí, hace eco del flujo

12. ERRORES COMUNES AL TRABAJAR CON I/O

1. System.FormatException

Causa: Usar `int.Parse()` o `double.Parse()` cuando la entrada del usuario contiene letras o está vacía.

Solución: Reemplazar siempre por `int.TryParse()` en código de producción.

2. Advertencia CS8600 (Nullability)

Causa: Ocurre porque `Console.ReadLine()` puede retornar `null` en flujos redirigidos o EOF.

Solución: Utilizar el operador de coalescencia nula (`?? ""`) o tipo `anulable (string?)`.

3. Conflictos de Coma / Punto Decimal

Causa: `double.TryParse()` respeta la cultura del SO. Si el sistema está en español espera coma (3,14), fallando si el usuario ingresa punto (3.14).

Solución: Usar `CultureInfo.InvariantCulture`.

4. Bloqueos Inesperados con ReadKey

Causa: Ejecutar aplicaciones con `Console.ReadKey()` en entornos automatizados sin consola interactiva asignada.

Solución: Validar `Console.IsInputRedirected`.

13. BUENAS PRÁCTICAS DE INGENIERÍA DE SOFTWARE

- **Nunca confíes en el usuario:** Valida defensivamente cualquier dato ingresado por teclado antes de pasarlo a la lógica de negocio.
- **Aplica siempre InvariantCulture para valores numéricos industriales:** Garantiza que los sensores o configuraciones que usan punto decimal no fallen en servidores con diferentes regiones.
- **Restaura los colores de la consola:** Si cambias `Console.ForegroundColor`, asegúrate de invocar `Console.ResetColor()` inmediatamente después de imprimir.

14. EJERCICIOS PRÁCTICOS DE APLICACIÓN

Ejercicio 1 – Comprensión Básica: Conversión e Interpolación

Solicita al usuario el nombre de un producto industrial y su precio unitario en formato `double`. Muestra en pantalla el resultado formateado con moneda (: C2) e incluye un IVA calculado del 16% utilizando interpolación de cadenas.

Ejercicio 2 – Aplicación: Bucle de Validaciones Numéricas

Construye un programa que pida al usuario su año de nacimiento. El programa debe repetir la solicitud mediante un bucle `while` hasta que el usuario ingrese un número entero válido comprendido entre 1900 y el año actual. Muestra la edad calculada.

Ejercicio 3 – Integración: Calculadora de Caudal Volumétrico

Crea una herramienta de consola que solicite el diámetro de una tubería (en metros) y la velocidad del fluido (en m/s). Calcula el área de la sección transversal ($A = \pi \cdot r^2$) y el caudal ($Q = A \cdot v$). Imprime los resultados con el formato de decimales fijos : F4.

Ejercicio 4 – Reto Avanzado: Menú Interactivo de Monitoreo

Implementa un menú de consola que utilice `Console.ReadKey()` para navegar entre 3 opciones sin requerir la tecla Enter:

- [1] Ver Temperatura de Motor
- [2] Ver Presión de Aceite
- [3] Salir del Sistema

15. MINI PROYECTO DEL CAPÍTULO

Proyecto: Sistema HMI de Supervisión de Calidad para Línea de Ensamblaje

Contexto: Una planta automotriz requiere una interfaz de consola interactiva para que un inspector de calidad registre el resultado de inspección de piezas en la línea de montaje en tiempo real.

Requisitos Funcionales del Sistema:

1. Limpiar la pantalla de la terminal y presentar un título en color amarillo con el nombre del sistema.
2. Solicitar el código del inspector (cadena no vacía) y el número de lote de producción.
3. Capturar la cantidad de piezas inspeccionadas (validando entero mediante `int.TryParse`).
4. Capturar la cantidad de piezas defectuosas encontradas en la muestra.
5. Calcular el porcentaje de defectuosidad ($\$Defectuosas / Inspeccionadas\$$).
6. Si el porcentaje de piezas defectuosas es mayor al 5.0%, mostrar una alerta en color rojo indicando **"LOTE RECHAZADO"**. De lo contrario, mostrar en verde **"LOTE APROBADO"**.
7. Utilizar especificadores de formato para mostrar el porcentaje de error con :P2 y la fecha/hora actual del sistema.

Pistas de Implementación:

Utiliza `DateTime.Now` para capturar la fecha del sistema, realiza la división asegurando convertir los valores a `double` para no perder decimales, y aplica `Console.ResetColor()` antes de finalizar la aplicación.

16. COMPRUEBA TUS CONOCIMIENTOS (AUTOEVALUACIÓN)

1. ¿Qué método de la clase `Console` captura una línea entera tipeada hasta presionar `Enter`?
a) `Console.Read()` b) `Console.ReadLine()` c) `Console.ReadKey()`
2. ¿Cuál es el tipo de dato devuelto por defecto por el método `Console.ReadLine()`?
a) `int` b) `object` c) `string`
3. ¿Qué ocurre si ejecutas `int.Parse("C#")`?
a) Retorna 0 b) Lanza una excepción `FormatException` c) Retorna null
4. ¿Qué método permite intentar la conversión de un texto a número retornando un `bool` sin romper la aplicación?
a) `int.Convert()` b) `int.TryParse()` c) `int.SafeParse()`
5. ¿Cómo se formatea el número `0.85` como `85.0%` usando interpolación de cadenas?
a) `($"{valor:P1}"` b) `($"{valor:C1}"` c) `($"{valor:F1}%"`

RESPUESTAS Y EXPLICACIONES

1. **Respuesta b:** `Console.ReadLine()` captura todo el flujo de caracteres ingresado hasta encontrar la tecla `Enter`.
2. **Respuesta c:** Retorna un `string?` (cadena que puede ser nula en ausencia de flujo de entrada).
3. **Respuesta b:** Lanza un error no controlado de tipo `FormatException` porque la cadena no tiene formato numérico.
4. **Respuesta b:** `int.TryParse()` realiza una conversión defensiva y asíncrona a nivel de error.
5. **Respuesta a:** El especificador `:P1` multiplica por 100 y agrega el símbolo `%` con 1 decimal.

17. RESUMEN DEL CAPÍTULO

Concepto	Para qué sirve	Sintaxis Clave	Error Común
StdOut	Imprimir información en la pantalla de la consola.	<code>Console.WriteLine(\$"...")</code>	Olvidar saltos de línea al formatear datos.
StdIn	Leer texto ingresado por el usuario vía teclado.	<code>string? s = Console.ReadLine()</code>	Asumir que el resultado es directamente numérico.
Parsing Seguro	Convertir cadenas a números sin provocar excepciones.	<code>int.TryParse(input, out int val)</code>	Usar <code>int.Parse()</code> sin bloques try-catch.
Interpolación	Formatear salidas complejas con especificadores.	<code>\$"{precio:C2}"</code>	Olvidar el símbolo \$ antes de las comillas.

10 Ideas Clave que Debes Recordar

1. Toda entrada capturada por teclado a través de la consola es de tipo `string`.
2. Usa siempre `int.TryParse()` o `double.TryParse()` para evitar caídas de software en producción.
3. `Console.Write()` mantiene el cursor en la misma línea; `Console.WriteLine()` avanza a la siguiente.
4. `Console.ReadKey()` es la mejor opción para pausar la pantalla o construir menús directos.
5. La interpolación de cadenas se activa mediante el prefijo `$` antes de abrir comillas.
6. El especificador `:C` aplica formato de moneda respetando la configuración del sistema operativo.
7. `Console.ResetColor()` previene que los colores aplicados afecten el resto de la terminal.
8. El stream de errores (`StdErr`) está separado del stream de salida estándar (`StdOut`).
9. El operador coalescente `??` evita advertencias de nulabilidad al capturar cadenas.
10. Aplica `CultureInfo.InvariantCulture` cuando proceses datos con formatos flotantes internacionales.

18. MAPA DE APRENDIZAJE PROMOCIONAL



Siguiente Paso: Ahora que dominas el intercambio de información entre el usuario y la memoria del programa, estás preparado para el **Capítulo 5: Estructuras de Control de Flujo**, donde aprenderás a tomar decisiones lógicas (`if`, `switch`) y repetir tareas mediante bucles (`for`, `while`).